



Empirical research on pair programming in higher education: a literature review

Anja Hawlitschek, Sarah Berndt & Sandra Schulz

To cite this article: Anja Hawlitschek, Sarah Berndt & Sandra Schulz (2022): Empirical research on pair programming in higher education: a literature review, Computer Science Education, DOI: [10.1080/08993408.2022.2039504](https://doi.org/10.1080/08993408.2022.2039504)

To link to this article: <https://doi.org/10.1080/08993408.2022.2039504>



View supplementary material [↗](#)



Published online: 06 Mar 2022.



Submit your article to this journal [↗](#)



Article views: 354



View related articles [↗](#)



View Crossmark data [↗](#)



Empirical research on pair programming in higher education: a literature review

Anja Hawlitschek ^a, Sarah Berndt^a and Sandra Schulz ^b

^aFaculty of Human Sciences, Otto-von-Guericke Universität Magdeburg, Magdeburg, Germany; ^bFaculty of Education, Universität Hamburg, Hamburg, Germany

ABSTRACT

Background and Context: Pair programming is an important approach to fostering students' programming and collaborative learning skills. However, the empirical findings on pair programming are mixed, especially concerning effective instructional design.

Objective: The objective of this literature review is to provide lecturers with systematic knowledge of current research evidences on the implementation of pair programming in courses by outlining the current state of knowledge on the effects of pair programming in higher education and on effective instructional design.

Method: The results are based on an analysis of 61 articles conducting empirical studies on pair programming in higher education published between 2010 and 2020.

Findings: Results of studies on the effects of pair programming in comparison with solo programming are to a great extent positive. Regarding instructional design there remain open questions, such as on effective instructional guidance and appropriate task design.

Implications: We highlight the need for more research on instructional design of pair programming to provide, for example, knowledge about effective procedures for pair programming, effective guidelines, or problem-solving approaches when problems occur within a team. We give recommendations for practitioners based on our findings.

ARTICLE HISTORY

Received 11 December 2020
Accepted 4 February 2022

KEYWORDS

Pair programming; literature review; instructional design

Introduction

Collaborative programming techniques like pair programming have been considered an important learning approach for the last two decades. Pair programming originates from agile software development. Its roots go back to the nineties, when it was described as part of extreme programming. Later, this method was adapted to teach programming, for example in computer science university courses. Pair programming has gained even more attention in the context of ongoing digital transformation, not only because it is claimed to foster novices' learning processes against the background of the increasing relevance of programming skills, but also because it supports the

CONTACT Anja Hawlitschek  Anja.Hawlitschek@ovgu.de  Otto-von-Guericke Universität Magdeburg, Magdeburg, Germany

 Supplemental data for this article can be accessed [here](#).

© 2022 Informa UK Limited, trading as Taylor & Francis Group

development of collaborative teamwork skills. Pair programming is by definition a collaborative activity (Bryant et al., 2006). In traditional pair programming scenarios, two students work side-by-side in front of a computer, programming together on the same code. However, today there also exist forms of pair programming where the programmers are locally distributed. The workflow is as follows: one student works as the “driver” who writes the code, while the other student has the role of the “navigator” and reviews each code line in the programming process. The roles are switched regularly. Pair programming as a collaborative learning activity is thus different from cooperative learning, which is characterized by a split of the workload between the participants – that is, each student works on their own piece of code which is put together at a specific time point to reach a final result. The idea behind the implementation of pair programming in computer science courses is, on the one hand, to prepare the students for real-world software development, which is often a collaborative activity (Umapathy & Ritzhaupt, 2017), and, on the other hand, to harness the proven benefits of collaborative learning in terms of learning outcome and motivation (Lou et al., 2001).

The integration of such collaborative learning activities in courses is demanding for lecturers, since in addition to the individual learner, the group dynamics and requirements for group coordination must be considered (Gillies & Boyle, 2010, 2011). Decisions on instructional design have to be made, for example on how to form pairs or how to guide students’ pair programming. However, the competencies of lecturers in structuring, organizing and monitoring collaborative learning are heterogeneous (Ruys et al., 2012). The specific implementation is often left to students and receives little guidance from lecturers, although thorough instructional design is essential for the effectiveness of teamwork (Gillies & Boyle, 2011).

Research objectives of our study

The goal of our systematic literature review is to support lecturers’ planning and implementation of pair programming in courses by presenting a systematic analysis of empirical research that concerns the application of pair programming in higher education in the last ten years. We examine not only studies comparing effects of pair programming versus solo programming and studies comparing instructional features, but also studies with other research objectives or approaches. This way, we analyze what additional factors were investigated in educational pair programming research to broaden the knowledge on empirical results on factors that are important for successful implementation in courses. Therefore, we did not restrict our analyses on studies of one particular type of empirical method. The main contribution of our study lies in providing an overview of recent empirical studies of pair programming in higher education, and in generating systematic knowledge about successful implementation for practitioners, for example, evidence-based instructional design principles.

We focus on the following research questions:

- RQ1: What is the current state of knowledge on the effectiveness of pair programming in comparison with solo programming?
- RQ2: What is known about effective instructional design of pair programming?

- RQ3: What additional factors are important for successful implementation of pair programming?

Effects of pair programming in comparison with solo programming

From meta-analyses, there is empirical evidence of the positive effects of pair programming compared to solo programming. These studies have overall reported positive effects of pair programming on performance (Dyba et al., 2007; Hannay et al., 2009; Salleh et al., 2011; Umapathy & Ritzhaupt, 2017) and persistence in computer programming courses or projects (Umapathy & Ritzhaupt, 2017). The role of prior skills in pair programming seems to be an important moderating factor here, insofar as pair programmers with fewer prior programming skills benefit most (Hannay et al., 2009). Also, task complexity is mentioned as potential moderator. Pair programming seems to be especially beneficial for performance on highly complex tasks while at the same time pairs invest a higher combined time effort (Dyba et al., 2007; Hannay et al., 2009). However, since these analyses cover research up to 2007 (Dyba et al., 2007; Hannay et al., 2009; Salleh et al., 2011) or to 2014 (Umapathy & Ritzhaupt, 2017), there is a need to investigate the current state of knowledge.

Instructional design of pair programming

There is little systematic knowledge from meta-analyses or literature reviews on effective instructional design for pair programming, which in fact is a base for effective learning (Hattie & Timperley, 2007). Reigeluth and Carr-Chellman (2009, p. 6) define instruction “as anything that is done purposely to facilitate learning”. Instructional design is both, a professional activity done by teachers and a discipline (Reigeluth, 1983). As professional activity, instructional design describes lecturers’ systematic didactic decisions with regard to the specific instruction, for example, in designing a course, a learning material or a lesson. In the design process, lecturers have to determine which method is best to reach the instructional goals under the specific instructional conditions. The discipline produces knowledge on methods of instruction as well as guidelines, for example principles for designing specific learning material, for structuring a task, or for guiding a learner to optimize instructional outcomes (see also, Gerjets & Scheiter, 2003; Sweller et al., 1998). Instructional methods are described as different ways to achieve different instructional outcomes under different conditions. Researchers also use the term “instructional feature” to describe the specific implementation of such a method in a learning environment (Mayer & Johnson, 2010). Instructional conditions are factors, like characteristics of content or learner characteristics, that influence effects of methods (Reigeluth, 1983).

For evidence-based didactic decisions, lecturers need systematic knowledge concerning which instructional features promote learning in pair programming scenarios. Important questions are for example: Which types of guidance are effective? What is the current knowledge on optimal pairing of students? How should a lecturer implement pair programming in the classroom?

Umapathy and Ritzhaupt (2017) outlined the significance of instructional support and guidance to help students to work according to the pair programming approach. This is a most important consideration from an educational point of view. That students work

collaboratively on tasks is not an added value per se for the learning process with regard to learning outcome or the development of teamwork skills. Collaborative learning is a complex activity, which requires a lot of cognitive and temporal resources, not least for the coordination of the group activities. Also, the quality of social interaction in collaborative learning is an important base for the success of the approach. If students successfully explain to each other and discuss their programming approaches and problems, this can highly support their learning (Van Boxtel et al., 2000). However, if the (perceived) cost of collaborative learning exceeds the (perceived) value, collaboration fails. Therefore, students – at least, novices – usually need instructional support to ensure the quality and success of collaborative learning activities (Nokes-Malach et al., 2015).

However, systematic analysis of the current knowledge landscape on instructional design of pair programming is scarce. One exception is the grouping of pairs, where results of a systematic literature review (Salleh et al., 2011) indicate that the method of pairing students according to compatibility in prior skills works most effectively.

It is important to improve knowledge about effective instructional features for pair programming and about the interrelation with learner characteristics or other instructional conditions. With this knowledge, lecturer can design more effective pair programming scenarios.

Method

We followed the procedure for literature reviews by Webster and Watson (2002) and the application by Viberg et al. (2018). We developed a review protocol according to Kitchenham and Charters (2007).

Literature search strategy

The literature search was carried out in the meaningful databases for computer science and interdisciplinary databases: Web of Science, ScienceDirect, ACM Digital Library, IEEE Xplore, SpringerLink, and WorldCat. A preliminary search with the string “pair programming” yielded more than 3000 hits. Therefore, we concluded that the specificity of our search was not high enough. Since we are focusing on higher education, where collaborative learning is a framework for the usage of pair programming, we integrated “collaborat*” in our search string. We applied subjective evaluation of performance of our search string. Pre-determined studies on pair programming in higher education that our search should identify were compared to the search results. We did not use “collaborative learning” because a number of primary studies already known, were not detected by the whole string, because they used terms such as “collaborative teaching” or “collaboration” in their abstracts, titles and/or keywords. The final search string thus was “pair programming” AND collaborat*. Additionally, we integrated “pair programming” AND cooperat* as search string for a second search, because the terms “collaborative” and “cooperative” are partly used interchangeable in literature (Olivares, 2008). In the databases, we searched in keywords, abstracts, and titles. Wildcards, Boolean operators, and the phrase search were linked so that the concrete search query was as follows: “pair programming” AND collaborat* respectively “pair programming” AND cooperat*. An initial search yielded 610 hits (not adjusted for duplications).

The first phase of selection conducted by one of our researchers was based on the following criteria. We only included those publications that had a thematic focus on pair programming in the context of higher education. In addition, articles must have been published in journals with peer-review procedures or in proceedings of conferences in the period from 1 January 2010 to 15 April 2020. In the latter case, only full papers were included, workshop papers or posters were excluded. Likewise, we only included empirical research studies – that is, pure tool or concept presentations were not considered (see criteria in [Figure 1](#)). Other quality criteria (see, Kitchenham & Charters, 2007) for inclusion in the final database were, whether the research objectives were stated, whether the participants, methods, and procedure were described and adequate, and whether study questions were answered. Quality criteria were only analyzed in the second phase of screening for those articles that were assessed for eligibility (see, [Figure 1](#)). If problems were identified in this phase, this was noted and discussed with the other researchers and a common decision on exclusion or inclusion was reached. For articles that had weaknesses but were selected for analysis nevertheless these weaknesses are recorded in the overview on included articles (see [Supplementary material](#)). Finally, the basis for the systematic literature review was 56 publications. A backward reference search was not performed.

Because of a time delay in the analysis process, we decided to perform a second literature search by the same researcher according to the procedure and criteria already described, to identify very recent studies in the analyzed databases, which were published between 15 April 2020 and 15 September 2020. This way, we were able to include three more articles in our analysis. Simultaneously, we searched in four additional databases for the whole time period (1 January 2010 to 15 September 2020) to broaden our data set, namely SSOAR, JSTOR, Fachportal Pädagogik, and SocioHub. The databases were selected because they are significant to sociology, pedagogy, as well as higher education research and, in this respect, bring another perspective to the topic of pair programming. We identified two additional articles. Our final data base therefore comprised 61 articles (36 conference papers and 25 journal articles).

Analysis of the studies

We analyzed the publications in order to assess the research in terms of methods of data collection, methods of data analysis and research objectives. The coding of the articles was done by one of the authors in a collaborative Excel spreadsheet. SPSS (version 27) was used for further statistical analysis. The first 15 (= 25%) articles (in alphabetical order) were coded by two researchers to validate and improve the coding process. Inter-coder reliability suggests an overall substantial agreement. Cohen's Kappa ranges from 0.74 for methods of data collection up to 1 for methods of analysis. In cases of different categorization, we discussed the differences and came to a joint decision.

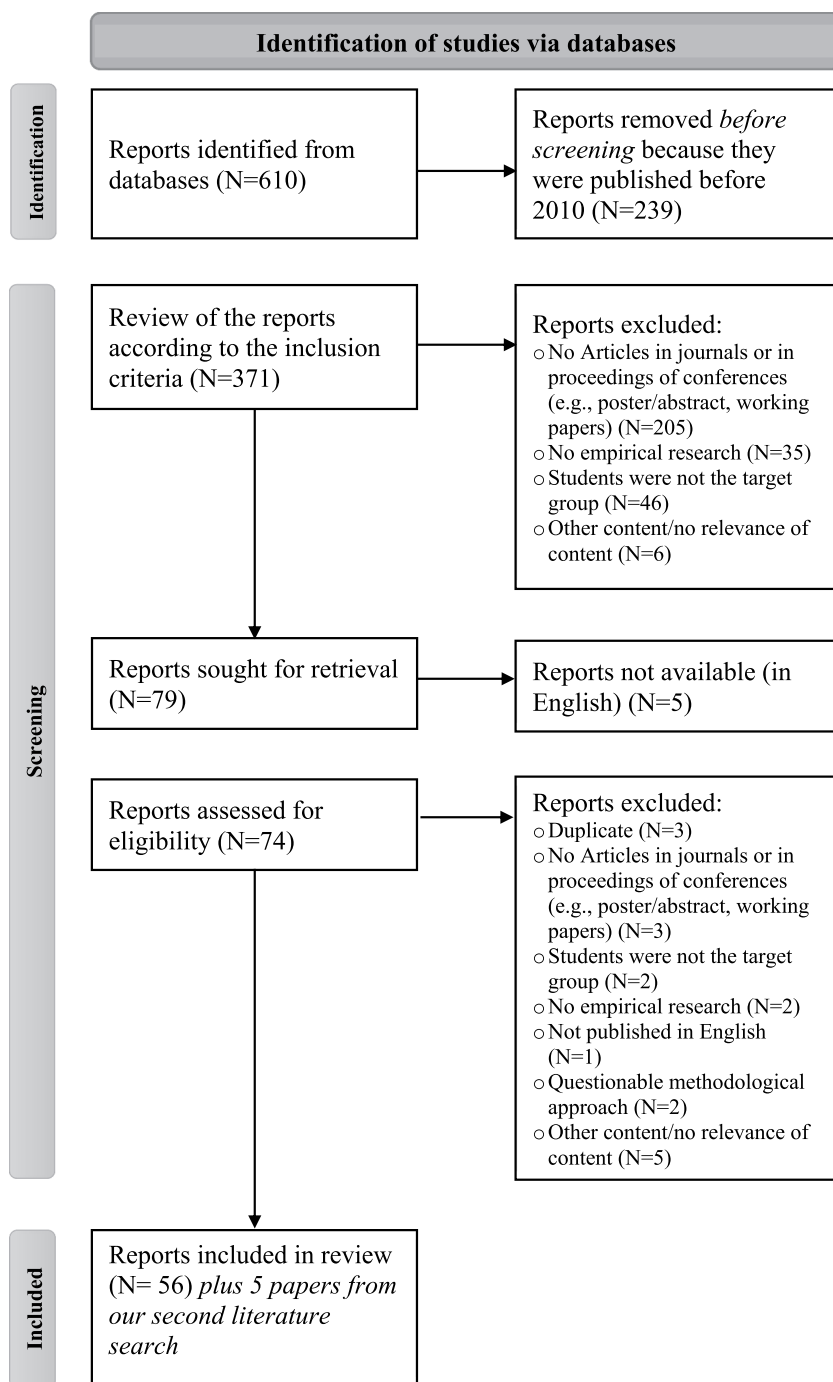


Figure 1. Flowchart for the Different Phases of the Systematic Review. *From:* Page et al. (2021)

Table 1. Methods of Data Collection.

Method	Description	N
Observation	Systematic registration of characteristics, events, and actions with sensory organs or technical sensors (e.g. observation log, videos, log files)	31
Qualitative interview/survey	Generation and recording of verbal expressions (e.g. unstructured or semi-structured interviews)	20
Quantitative survey	Standardized questionnaires	51
Achievement test	Assessment of learning outcomes/performance (e.g. scores for programming assignments, exam grades, scores for code quality)	38
Physiological measurement	Recording and quantification of physiological processes with measuring devices (e.g. EEG)	0

Note: Some studies applied more than one method of data collection, therefore, the total number is >61.

Methods of data collection and analysis

We adopted categories for methods of data collection from Döring and Bortz (2016). We focused on five methods (see, Table 1). Other methods like psychological assessments turned out to be irrelevant for our sample. After noting in the Excel spreadsheet precisely which data were collected, we categorized them accordingly.

We categorized the studies according to their methods of data analysis as qualitative, quantitative, or mixed methods (Döring & Bortz, 2016). Studies which applied both qualitative and quantitative methods of data analysis were coded as mixed methods if the partial studies did not stand independently of each other but were integrated.

Research objectives of the studies

To get an overview of the research objectives, we recorded the criteria applied. Salleh et al. (2011) categorized measures of effectiveness, which we could use as a basis. However, since, we considered also variables that were not measures of effectiveness (such as learning behavior), we adapted our categories of criteria accordingly (see, Table 2). We also recorded the independent variables, if existing. For studies without independent variable, we recorded research goals. Based on these results, we categorized the studies (see also, Mayer (2011), for a similar approach) in (1) studies that compare effects of solo versus pair programming, (2) studies that analyzes the effects of adding an instructional feature (i.e. implementing a specific instructional design) to pair programming in comparison with a condition

Table 2. Categories of Criteria.

Category	Example	N
Programming performance	Number of solved programming tasks, quality of code	32
Students' perceptions	Satisfaction, motivation, perception of compatibility	36
Learning outcome	Grade, measurement of skills	22
Perceived learning outcome	Self-rating on learning outcomes	10
Persistence	Drop-out rate	3
Efficiency	Completion time	6
Programming behavior	Code contribution, number of lines of code, number of comments	11
Learning behavior	Communication with other students, support of each other, attendance	18

Note: Some studies applied more than one criterion, therefore the total number is >61.

without this feature – for example, a specific composition of pairs versus randomly paired students – or with a condition with a different implementation of that feature – for example, homogenous pairs versus heterogeneous pairs –, and (3) studies that investigate pair programming with other research objectives and approaches.

Results

In the following chapters, we describe results from our analyses as descriptive synthesis (Kitchenham & Charters, 2007). We use direction of effect as standardized metric for RQ1 and RQ2 and provide effect direction plots where applicable (Campbell et al., 2020). The visualization of effect direction plots is based on Boon and Thomson (2021).

Methods of data collection and analysis

The most commonly applied method (see, Table 1) was the quantitative survey (N = 51). Achievement tests were implemented in 38 studies, mainly using assignment scores (e.g. Zacharis, 2011) or exam grades (e.g. Rong et al., 2012), but also other assessment tests like multiple-choice tests (e.g. Rodriguez & Boyer, 2015). Observations were applied in 31 articles; mainly the researchers used log files for their analyses (e.g. Estácio, Oliveira et al., 2015; Zacharis, 2011). Qualitative interviews/surveys were also applied, but only in 20 articles. Some limitations have to be mentioned concerning the studies that are classified here as having applied qualitative interviews/surveys. These surveys were mainly merely open-ended questions on experiences and perceptions of pair programming, which were part of a quantitative survey (e.g. Benadé & Liebenberg, 2017). Only one article, investigating gender differences in the perceptions and experiences of pair programming (Ying et al., 2019), applied a genuine qualitative approach with qualitative research questions and a qualitative research instrument without using also quantitative data. Not a single study applied physiological measurements.

Concerning the methods of data analysis, there was a relatively clear focus on quantitative analysis that was applied in 51 of the studies. Five studies applied qualitative methods of data analysis. Nine studies applied a combination of quantitative and qualitative analysis, according to our definition of mixed methods. Studies which only cited some example sentences from their open-ended questions to underline arguments but did not describe their procedure for analysis of qualitative data were not categorized as using qualitative analysis methods (e.g. Celepkolu & Boyer, 2018a). This is the reason for the discrepancy between the number of studies using qualitative interviews/surveys and the number of studies, which applied qualitative analysis of their data.

Research objectives of the studies

The criteria the studies applied differ greatly (see, Table 2). Effects on students' perceptions were most commonly researched. The effects on programming performance and learning outcomes were also researched widely. A moderate number of research studies

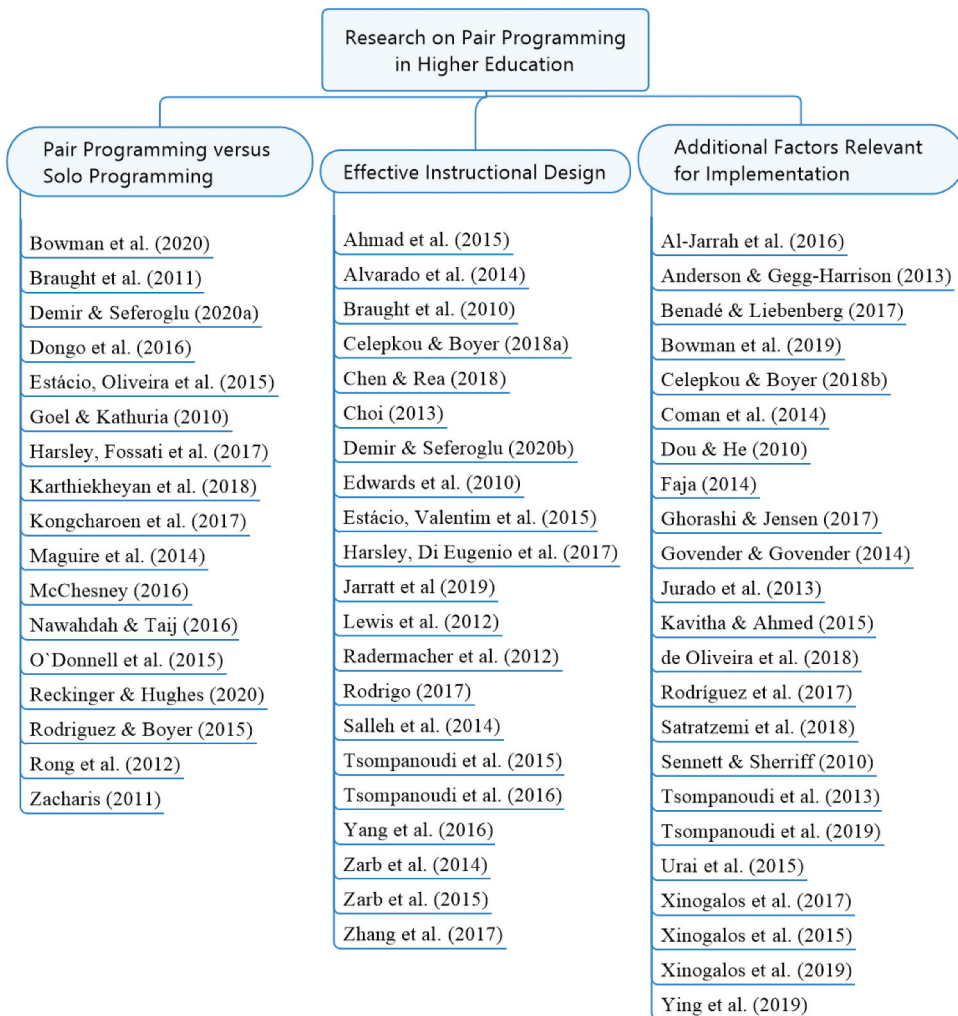


Figure 2. Overview on Categorization of Studies with Regard to Research Objective.

considered the effects of pair programming on learning behavior or programming behavior or perceived learning outcome. A small number of studies focused on efficiency and persistency.

A comparison of pair programming and solo programming approaches was applied by 17 studies. Research on effective instructional design was conducted by 21 studies. Most of these investigated the effects of instructional features within a pair programming format ($N = 19$). We also identified studies that analyzed pair programming versus distance pair programming ($N = 2$). Although not pertaining to instructional design in a narrower sense, it can be a didactic decision to apply a specific form of pair programming (like distributed pair programming). For this reason, we include these studies into this category. Twenty-three studies investigated and evaluated pair programming with a research objective or approach different than the first two categories (see, Figure 2).

Research evidence on pair programming

Effects of pair programming in comparison with solo programming

The empirical evidence that pair programming improves learning processes in higher education in comparison with solo programming is mixed. In our literature review, a majority of studies show positive results. These studies reported positive effects of pair programming in comparison with solo programming for virtually all categories of dependent variables (see, Table 3). Better programming performance for pairs in assessments (Demir & Seferoglu, 2020a; Estácio, Oliveira et al., 2015; Reckinger & Hughes, 2020; Rong et al., 2012; Zacharis, 2011) and better code quality, for example, fewer errors (Estácio, Oliveira et al., 2015; Harsley, Fossati et al., 2017; Kongcharoen et al., 2017; Nawahdah & Taji, 2016; Rong et al., 2012; Zacharis, 2011), were reported most often.

Table 3. Effects Direction Plot for Studies Analyzing Pair Programming in Comparison with Solo Programming.

Study	Programming performance	(Perceived) learning outcome	Students' perceptions	Learning behavior	Programming behavior	Persistence	Efficiency
Bowman et al. (2020)		▼					
Braught et al. (2011)	◄► mixed	◄► no	▲			▲	
Demir & Seferoglu (2020a)	▲		▲				
Estácio, Oliveira, et al. (2015)	▲		▲				
Karthiekeyan et al. (2018)	◄► mixed						
Kongcharoen et al. (2017)	▲	▲		▲	▲		
Maguire et al. (2014)		◄► mixed		◄► no			
McChesney (2016)	◄► mixed		◄► no				
O'Donnell et al. (2015)	◄► no			◄► mixed			
Reckinger & Hughes (2020)	▲						
Rodriguez & Boyer (2015)		▼					
Rong et al. (2012)	▲	▲					◄► no
Zacharis (2011)	▲	◄► no			▲		
Dongo et al. (2016)+	◄► no		▲				
Goel & Kathuria (2010)+	◄► mixed	▲					
Harsley, Fossati et al. (2017)+	▲	◄► no	◄► no	▲	▲		▲
Nawahdah & Taji (2016)+	▲			▲	▲	▲	

Note: effect direction PP: ▲ = positive effect, ▼ = negative effect, ◄► = no differences/mixed effects; sample size: ◄► < 50, ◄► 50–150, ◄► > 150; study quality: + indicates some concerns

Studies also reported higher grades or exam scores (Kongcharoen et al., 2017; Rong et al., 2012) and – however, only reporting descriptive data – better problem solving for pairs (Goel & Kathuria, 2010). Students in pair programming conditions reported more confidence in their results (Braught et al., 2011; Dongo et al., 2016), more motivation (Estácio, Oliveira et al., 2015), and more flow (Demir & Seferoglu, 2020a). They were less prone to drop-out (Nawahdah & Taji, 2016) and had a higher probability of successful course completion (Braught et al., 2011). Pairs needed significantly less time to complete problems and they were more efficient in programming than solo programmers (Harsley, Fossati et al., 2017). Their programming outcomes differed from solo programmers in terms of a smaller number of statements, more classes, more comments (Nawahdah & Taji, 2016), fewer undos and redos (Harsley, Fossati et al., 2017), and more lines of code per hour (Zacharis, 2011). Pairs showed more activity in programming (Kongcharoen et al., 2017). Also, their learning behavior was different. Pairs requested significantly fewer system-provided examples, used a lesson tutorial significantly longer than individuals (Harsley, Fossati et al., 2017), and had fewer questions for lecturers (Kongcharoen et al., 2017). Because of the collaborative nature of pair programming, observations showed significantly less typing and gaze-off, as well as significantly more talking and smiling (Nawahdah & Taji, 2016).

However, some studies found mixed results indicating that pair programming is beneficial only for specific tasks or user groups. Studies examining pair programming for specific tasks showed that grades are higher for pair programming conditions in specific assignments (Dongo et al., 2016) or that pair programming leads to higher programming performance in some phases of programming (e.g. elaboration, code construction; Karthikeyan et al., 2018). Studies examining pair programming for specific user groups point out that pair programming is especially effective for the programming performance of students with lower prior programming skills (Braught et al., 2011; Goel & Kathuria, 2010; McChesney, 2016). There is also evidence that pair programming leads to a higher level of attendance among weaker students (O'Donnell et al., 2015) and better grades for female students (Maguire et al., 2014). In all of these studies, the positive effects of pair programming were limited to the particular user group or a particular task.

There are also studies that found no improvement at all in comparison with solo programming; however, pair programming was generally equally effective. These studies found no significant differences in programming performance (Dongo et al., 2016; O'Donnell et al., 2015) or learning outcome (Harsley, Fossati et al., 2017; Zacharis, 2011). Furthermore, in some studies, time to finish assignments (Rong et al., 2012) and students' attendance (Maguire et al., 2014) did not differ between groups. Participants also reported no significant differences in enjoyment (Harsley, Fossati et al., 2017; McChesney, 2016).

Two studies reported effects not in favor for pair programming. In one study, students in the pair programming condition did not significantly improve their learning outcomes in comparison with a pre-test while solo programmers did (Rodriguez & Boyer, 2015). One large-scale study with more than 1,500 students found negative short-term and long-term effects of pair programming. In fact, students in the pair programming condition showed a reduction in grades, as well as in their subsequent engagement with computer science courses and the likelihood of majoring in computer science. This effect especially affected white students who were paired with students with either low or high average levels of prior experience with computer programming (Bowman et al., 2020).

Table 4. Effects Direction Plot for Studies Analyzing Pairing

Study	Confidence level	Homogenous programming skills	Homogeneous mental model consistency	Same gender pairing	Heterogenous personality traits	Homogenous learning style	Friendship
Alvarado et al., 2014	◄► no						
Brought et al., 2010		◄► mixed					
Demir and Seferoglu (2020b)		◄► no		◄► no	◄► mixed	◄► mixed	◄► mixed
Radermacher et al., 2012			◄► mixed				
Choi (2013)				◄► mixed			
Chen & Rea, (2018)				◄► no			
Jarratt et al. (2019)				◄► mixed			
Salleh et al., 2014					◄► no		
Zhang et al. (2017)++					▲		

Note: effect direction: ▲= positive effect, ▼= negative effect, ◄►= no differences/mixed effects; sample size: ◄► < 50, ◄► 50-150, ◄► > 150; study quality: ++ indicates serious concerns

Effective instructional design of pair programming

A majority of studies in this category examined the effective pairing of students. They analyzed whether pairing methods based on students’ confidence, prior skills, gender, consistency of mental models, or personality traits led to improved performance. As can be seen in Table 4 the findings are mixed.

Whereas grouping according to confidence level had no influence on performance (Alvarado et al., 2014), the other factors at least partly affected performance. To match students with similar levels of programming skills or experience levels rather than making random assignments was beneficial for programming performance of less able students (Brought et al., 2010). The pairing of homogeneous groups where the members exhibited mental model consistency led to better programming performance in some exams (Radermacher et al., 2012). However, Demir and Seferoglu (2020b) could not find any effect of pairing according to prior knowledge on outcome variables like perceived group compatibility, flow, or programming performance.

There are also mixed results concerning gender pairings. Choi (2013) found no significant differences in programming performance (see, also Chen & Rea, 2018; Demir & Seferoglu, 2020b) or problem-solving ability. However, same-gender pairing led to significantly higher communication and compatibility levels (Choi, 2013). The results of Jarratt et al. (2019) indicate a benefit of pairing female students with female students, these pairs completed a significantly greater proportion of assignments. The results concerning pairing according to personality traits are mixed. Conscientiousness and neuroticism do not appear to be relevant for performance (Salleh et al., 2014). However, Demir and Seferoglu (2020b) found that heterogeneous groups concerning the conscientiousness component of personality traits led to a higher level of coding

performance. Y. Zhang et al. (2017) analyzed combinations of introvert, extrovert and neutral students and found a slight benefit for mixed personality pairs in test performance, however this descriptive result have to be interpreted with care due to flaws in the article (see [Supplementary material](#)). Furthermore, variables like friendship or learning style might be important for pairing. Whereas pairs consisting of friends experienced greater flow (the same holds true for homogeneous learning style), friendship pairs went hand in hand with a lower level of coding performance (Demir & Seferoglu, 2020b).

Three studies examined instructional guidance in pair programming. The results of these studies indicate different effects of guidance for learning in pairs, depending on the instructional design of the guidance and educational criteria. One of these studies was on feedback for the pair programming activity. Students in one condition received automated feedback by visualization of group performance and participation, while the other condition pair programmed without such feedback. Whereas students in the two conditions were equivalent in their coding efficiency, pairs with feedback exhibited better planning in difficult problem-solving situations (Harsley, Di Eugenio et al., 2017). The other two studies examined guidelines for novice pair programmers' communication (e.g. "do not hesitate to ask your partner for clarification"), learning strategies (e.g. "look back on your last couple of steps and review your previous work"), and pair programming activities (e.g. "while the driver is coding, actively look to make suggestions that contribute to the code"). In fact, this kind of guidance supported students' positive perception of the pair programming experience (Zarb et al., 2014). Whereas Zarb et al. (2014) found no significant differences concerning the number of tasks solved, communication in the guideline condition was perceived to be significantly easier and the contribution of each partner was rated higher (see also, Zarb et al., 2015).

The procedure of pair programming was analyzed in three studies. All of them investigated the role switches between pairs, but with mixed approaches and mixed results. Tsompanoudi et al. (2015) compared two conditions – an independent switch group, where the pairs arranged the driver and navigator roles themselves, and a system-based switch group where a computer system initiated role switches based on a balanced knowledge acquisition policy. The system role switches led to a significantly higher number of role changes and a significantly better task distribution; however, there were no significant differences in students' contributions (ratio of the typed code to the total code), completion time, and performance. In a second study, a computer system assigned students as driver or navigator, either alternating or on the basis of prior skills. In one condition, there were successive role alternations, so that pairs switched roles after each task. In the other condition, the switch was based on students' individual skills. Therefore, the system assigned the driver role to the student with the least experience in the area of the task (Tsompanoudi et al., 2016). The findings suggest that the two strategies for role distribution were equally effective. In the third study, a condition without role switch and a condition with role switch were compared (Ahmad et al., 2015). The results indicate that pair rotation improved programming quality, however, no tests of significance were reported.

Two studies compared delivery modalities – that is, traditional co-located pair programming versus hybrid pair programming (Celepkolu & Boyer, 2018a; Edwards et al., 2010). Results indicate that students prefer co-located pair programming (Celepkolu & Boyer, 2018a; Edwards et al., 2010). Regarding learning gain, no differences were found (Celepkolu & Boyer, 2018a).

One study examined a highly collaborative versus a less collaborative condition. In the highly collaborative condition, the students collaborated all the time; in the less collaborative condition, they solved two tasks alone and collaborated only on the last task. The results indicate that students with low prior skills benefit in terms of learning outcome from the highly collaborative condition, especially females (Lewis et al., 2012).

The role of the audience was examined in one study. Descriptive analyses suggest that pair programming without an audience was perceived as more beneficial for learning (Estácio, Valentim et al., 2015). The effects of using different kinds of annotations were studied by Yang et al. (2016). The control group had to add algorithmic description comments to the beginning of programs, the experimental group added description comments to the data structures of programs line by line. The latter reported significant higher retention. In a further study, the influence of the instructional design of tasks on communication in groups was examined. One group had to identify bugs in code (static condition), the other group had to identify and solve bugs (dynamic condition). High performing students in the static condition exchanged more words in the programming process than students in the dynamic condition (Rodrigo, 2017). However, it is not clear from the article what might be the cause and the implications of this somewhat counter-intuitive effect.

Additional factors that are important for successful implementation of pair programming

Studies in this category predominantly evaluated the application of pair programming and investigated learners' experiences and attitudes toward pair programming. Because several of the studies applied purely descriptive statistics (see [Supplementary material](#)), the following results have to be interpreted with great care.

Students in these studies overall rated pair programming as a useful learning technique that improves programming abilities and communication skills (e.g. Anderson & Gegg-Harrison, 2013; Benadé & Liebenberg, 2017; Govender & Govender, 2014; De Oliveira et al., 2018; Xinogalos et al., 2015). However, high-achieving students reported fewer benefits of pair programming (Celepkolu & Boyer, 2018b). Furthermore, in a study of Ying et al. (2019) men and woman had different perceptions of benefits and challenges of pair programming, for example, only woman spoke about the fear of burdening their partner in pair programming whereas only men expressed the feeling that their partner was burdening them.

Pairing of students was also investigated here. The pairing with more experienced programmers was related with a lower perceived learning outcome and less time as driver (Bowman et al., 2019). On the whole, students seem to prefer to work in pairs with similar programming skills (Celepkolu & Boyer, 2018b; Sennett & Sherrieff, 2010) and similar time management (Dou & He, 2010). However, Xinogalos et al. (2019) found no statistical

association between pair compatibility on prior skills and performance. Furthermore, Xinogalos et al. (2017) point out in their study that the most important criterion for students' group formation was friendship, not the same level of programming knowledge.

Also frequently analyzed were (effects of) learner characteristics, perceptions or behavior in pair programming. In two studies, students' performance was associated with their experience and confidence in programming rather than on how comfortable they felt during pair programming (Satratzemi et al., 2018; Tsompanoudi et al., 2019). Also, students' level of experience with pair programming was important for their (perceived) performance (Faja, 2014). The communication of pairs seems to be most relevant for outcomes. Rodríguez et al. (2017) found that students' task-related dialogue (e.g. providing feedback to their partner, asking questions) was interrelated with quality of coding and learning gain. However, unequal participation rates of the partners in programming were identified as common and as a potential threat to learning (Tsompanoudi et al., 2013). The extent to which students perceived their partners as equally contributing, also influenced students' perceptions of other factors. Increasing equality in contribution was positively associated with perceived learning, confidence in work quality, enjoyment and productivity (Faja, 2014). Furthermore, factors like perceived usefulness of pair programming were positively related with perceived learning, satisfaction and intention of adoption in future (Kavitha & Ahmed, 2015). Factors hindering student collaboration were primarily technical problems and coordination in terms of a common time to collaborate (Xinogalos et al., 2017).

Features of pair programming tools were also researched (Al-Jarrah et al., 2016; Ghorashi & Jensen, 2017; Jurado et al., 2013), for example, which features are important for students in distributed or in co-located pair programming (Urai et al., 2015).

The observational study by Coman et al. (2014) sheds light on the use of pair programming techniques in practice of higher education and industrial settings. The results indicate that pair programming is mostly applied for dealing with complex tasks or rather parts of an issue where intensive interaction seems to be productive, for example, in order to get unstuck, share knowledge or overcome difficulties in programming. Thus, pair programming is rarely used exclusively as a general development technique to solve a task but often combined with types of cooperative teamwork and solo sessions. Moreover, the concrete application of pair programming did most often not strictly adhere to the formal definition. Coman et al. (2014) conclude that the nature of tasks should be considered more carefully in research on pair programming to avoid a bias of results. The benefits of pair programming seems to increase with the increasing need to collaborate to solve a task successfully.

Discussion

Our study presented a literature review of research on the use of pair programming in higher education. We focused our analyses on the potential effects of pair programming in comparison with solo programming, on effects of adding an instructional feature, and on additional factors important for implementation of pair programming.

Methodologically, studies with quantitative data collection and analysis predominate. There is only one genuine qualitative research study with sound qualitative methods. The application of more such studies would help to find out more about fine-grained interactions in pair programming and the underlying mechanisms of learning.

The most important factors under focus in higher education pair programming research are students' perceptions, along with programming performance and learning outcomes. We assume students' perceptions to be so prominent here, because they can be easily surveyed. Most studies used an easy to apply questionnaire here. Surprisingly, in our study, only a few studies focus on persistence. Because of the relatively huge number of dropouts from computer courses, especially among novices (Kori et al., 2016, 2015), this factor should be addressed more in further research. In the following chapters, we discuss our results and derive recommendations for lecturers, also with reference to other empirical research on collaborative learning.

Effects of pair programming in comparison with solo programming

Concerning RQ1, we found evidences for positive effects of pair programming in higher education. A majority of studies in this category reported a benefit of pair programming in comparison with solo programming, especially in terms of programming performance. This result correlates with the results of the meta-analyses by Hannay et al. (2009) and Umapathy and Ritzhaupt (2017).

However, the negative results from Bowman et al. (2020) point on the importance of ongoing research on the effectiveness of pair programming to reveal under which circumstances pair programming is ineffective.

In their meta-analysis, Hannay et al. (2009) also found a significant positive overall effect of pair programming on duration and a significant negative overall effect on effort. In our literature review, studies did not observe those outcomes quite as often. Studies which examined such effects either found no effects at all (Rong et al., 2012) or they found a reverse effect, in that pair programming groups needed significantly less time and were more efficient (Harsley, Fossati et al., 2017). However, the literature review by Salleh et al. (2011) supports our findings. Hence, the effect of pair programming on productivity remains unclear.

Another important result of meta-analyses and literature reviews (Hannay et al., 2009; Salleh et al., 2011) is seen in the role of the prior skills and experiences of pair programmers. Pair programmers with fewer prior skills and experiences benefit more from the pair programming approach. This result is reflected in our data (Braught et al., 2011; Lewis et al., 2012; McChesney, 2016; O'Donnell et al., 2015). It seems to be the case that novice pair programmers especially benefit from the exchange of ideas and reflection on the code in the pair. While programming with a partner, they might get less easily lost in their own code (Hannay et al., 2009). Consequently, pair programming can be a valuable method in novice programming courses in higher education.

Effective instructional design of pair programming

Our research question on the instructional design of pair programming described as effective (RQ2) is harder to answer. Studies focus mainly on questions concerning the building of pairs. Other areas of research on instructional design, for example, regarding the implementation of guidance for successful pair programming, are underrepresented. However, guidance is important for both learning programming and collaborative learning. Novice students who learn to program without any type of instructional guidance usually have problems understanding the underlying concepts and perform worse than students who learn with guidance when moving on to other problems (e.g. Mayer, 2004). Collaborative learning, on the other hand, which goes hand in hand with the challenges and hindrances already described, is a kind of learning activity which likely fails when students have little experience or low skills in collaborative teamwork (Kreijns et al., 2003). Therefore, it is important to improve students' collaboration skills to make teamwork more effective and to provide guidance for their collaborative learning activities (Zarb et al., 2013).

Our results concerning the pairing of students are mixed. We did not find a result supporting the pairing of students with heterogeneous prior skills. Results indicate, that it might be better to pair students with homogeneous level of programming skills or experience, especially with reference to similar results from Salleh et al. (2011). However, from our results it is not clear whether this only pertains to students with lower abilities (Braught et al., 2010). In the literature, we found some explanations for benefits of homogeneous groups in this regard. One reason could be that in mixed pairs, programming experts lack adequate knowledge of how to explain programming concepts to novices (Demir & Seferoglu, 2020b). In this case, students would need instructional guidance, such as the implementation of guidelines on how to explain and how to handle problems (Zarb et al., 2013). Another reason could be that students with low prior knowledge depend too much on their expert counterpart as result of reduced self-confidence because of being the novice, or simply from resource management and thus have lower learning outcomes (Bowman et al., 2019). Given the weak evidences from our study for homogenous groups and the importance of skills in dealing with heterogeneous people in the later job, lecturer from courses whose learning objectives also include teamwork skills should carefully consider the benefits of each approach. However, benefits for heterogeneous pair programming teams in learning to collaborate, where not object of research in the studies analyzed.

Friendship of pairs is not a necessary precondition for successful collaboration. Students seem to feel more comfortable and have sincere communication when pair programming with friends, and have to invest less mental effort or fewer time resources to organize the collaboration; however, there also could be more non-task interaction and thus lower performance (Demir & Seferoglu, 2020b). The mixed results on pairing according to gender or personality traits is congruent with the results of Salleh et al. (2011). In a meta-analysis on teamwork, Bowers et al. (2000) point out that the complexity and difficulty of tasks could be a moderator for these effects. Homogeneous teams seem to perform better on low difficulty tasks, whereas complex tasks are better handled by heterogeneous teams. As a reason for this,

Bowers et al. (2000) assume that to solve complex, difficult tasks requires a higher amount of creativity, which is usually found in heterogeneous teams. Since decisions about grouping can depend on the complexity of tasks a possible approach for programming courses could be to bring homogeneous partners together in novice courses and change the grouping to heterogeneous teams in advanced courses.

Furthermore, our results suggest that role rotation (i.e. a regular switch between driver and navigator) is important for the pair programming process and to achieve an optimal learning outcome. However, the optimal procedure remains unclear. System interventions in turn-taking did not have the expected positive effects on students' participation rates or performance (Tsompanoudi et al., 2015). The same holds true for guidance of pair programming. Therefore, we highlight the need for more knowledge about effective procedures for pair programming, effective guidelines for pair programming, and problem-solving approaches when problems occur within a team. We need evidence-based instructional design principles in particular for different levels of prior skills, but also in relation to other learner characteristics.

Additional factors that are important for successful implementation of pair programming

Regarding RQ3, we consider the following as additional factors that are important for successful pair programming implementation: friendship of pairs as students' preferred selection criterion (Xinogalos et al., 2017), fears regarding pair programming (Ying et al., 2019), perceived usefulness of pair programming (Kavitha & Ahmed, 2015), unequal participation rates (Tsompanoudi et al., 2013) as well as meaningful communication (Rodríguez et al., 2017).

Concerning the pairing of students, results support the recommendation to pair students with similar programming skills (Celepkolu & Boyer, 2018b; Sennett & Sherriff, 2010) in particular for less-experienced programmers (Bowman et al., 2019). However, results also indicate that students mainly choose friends for teamwork (Xinogalos et al., 2017). As we already discussed in the preceding chapter based on results from Demir and Seferoglu (2020b) friendship is not the best criterion for selection if optimal performance is the goal. To assign student pairs by the lecturer or a system on the basis of a prior knowledge test thus can be beneficial for learning but also might induce resentment if students are accustomed to work together with their friends. From empirical studies it is known that lecturers' clarity on objectives and policies of the course as well as on students' responsibilities has a positive effect on achievement (Schneider & Preckel, 2017). To clarify such instructional design decisions for students, like criteria applied for pairing, might help to avoid potential sources for discontentment.

The reported fears regarding pair programming (like burdening the partner or being burdened by a partner) but also the importance of perceived usefulness points on the need to clarify the value of working with a pair especially if it is a pair with lower prior skills. In the context of Expectancy-Value Theory, Eccles et al. (1983) highlight the importance of perceived value for learning motivation. Lecturer can increase students' perceived value of pair programming for example, by informing about the usefulness for

further study or professional work, in fact, teamwork skills are described as one of the most important skills for software engineers, which graduates often lack (Garousi et al., 2019).

It is also important to prevent unequal participation rates of the pairs in programming, for example, that students with low prior knowledge are too passive and write considerably less code (Tsompanoudi et al., 2013) – not only because of the risk of lower performance and learning for the less participating student, but also because of the importance of an equal contribution for students' perceptions (Faja, 2014). Concerning the latter, research on collaborative learning highlight the importance of positive interdependence in groups. Positive interdependence is reached when the partners in teamwork depend on each other, when each partner contributes a fair share to the group work, and when group success can only occur when all group members succeed (Johnson & Johnson, 2009). To ensure the application of pair programming in a course according to the pair programming definition, for example, by introducing roles, requirements, and procedure at the beginning of a course, is a first step to achieve this. In a second step, monitoring and (if necessary) guidance of students' pair programming can be implemented. The usage of log files to visualize the behavior of each group member (e.g. in a dashboard for the team) can therefore be considered. In particular, to provide a possibility of comparison with good performing groups, for example, behavioral patterns of a successful collaborating group as best practice example, helps students to adjust their activities in teamwork (Jermann & Dillenbourg, 2008).

Findings also reveal that meaningful communication between partners is important for outcomes (Rodríguez et al., 2017). That meaningful communication and even more the quality of students' interaction is beneficial for learning is also known from other studies about collaborative learning (Kaendler et al., 2015). Clear instructions on how to collaborate and communicate (Saab et al., 2007) or an exemplary model of collaboration (Rummel & Spada, 2005) support students in this regard.

Results of Coman et al. (2014) highlight the importance of appropriate task design for pair programming, which was also a result from Hannay et al. (2009), respectively, the usage of pair programming only for tasks where collaboration is a benefit. Further research is needed to analyze the impact of task complexity on pair programming performance in more detail. Coman et al. (2014) also describe a simplified application of the formal pair programming procedure in practice. It is not clear, whether this is due to a lack of time or a lack of appropriate skills and knowledge. However, the latter possibility points once more to the need that lecturers thoroughly introduce students to the pair programming procedure and goals.

Recommendations

Our results make clear that pair programming is not easy to apply. The instructional design is especially important for success. Pair programming is a challenging learning activity. There are strong dependencies between the team members, and the organization can be very time-consuming. Given the current state of knowledge, we derive – with due care – some recommendations for practitioners to enable successful pair programming in courses: (1) Tasks should be designed in a way that collaboration is a benefit for students. Appropriate tasks are neither too brief nor too easy. (2) Pairs have to collaborate

in accordance with the pair programming approach – for example, switch roles, communicate with each other meaningfully. Lecturers should not leave the actual way of implementing pair programming to their students alone. At the beginning of the course or task, they should provide guidelines for pair programming, give an introduction into pair programming, and clarify the value of the approach. (3) Pairs should have the same level of prior experience and programming skills or else how students should handle heterogeneity should be part of instruction. Lecturers should also explain the causes for chosen pairing methods. (4) If students are novices, lecturer should monitor the process of pair programming, to identify groups with problems in teamwork early on and to ensure equal participation rate and meaningful interaction. To support lecturer in monitoring, analysis techniques from learning analytics can be used (Cukurova et al., 2020). With these, it is possible to identify patterns of user behavior that indicate good collaboration respectively teams at risk of underachieving or even dropping out. The application of learning analytics in pair programming can support not only lecturers but also students. Results from learning analytics could be used for feedback for students – for example, informing students about their performance relative to best practice, or giving them guidance. Lecturers, on the other hand, could get information on which teams need support or which task design is suboptimal.

Limitations

The limitations of our literature review result from our methodological and practical decisions but also from the quality of the analyzed studies. First of all, our findings on pair programming may be affected by bias, as studies without significant effects may have been published on a smaller scale. This phenomenon, known as publication bias, should always be considered. We also could not integrate literature that was not published in English. The restriction of our study to data from the last ten years was because of our interest in actual development of the research area, but also due to practical considerations regarding a manageable number of publications. This way, important information from earlier research was not considered. However, given that Salleh et al. (2011) analyzed 1999 to 2007, with a slightly different focus, the loss might be tolerable.

Furthermore, we did not apply manual literature search. Performance of our search thus is highly connected with quality of the search string (H. Zhang et al., 2011). Since we applied subjective evaluation of performance of our search string, the reliability only depends on experts' knowledge. A better approach for further systematic literature reviews would be to apply the systematic literature search process proposed by H. Zhang et al. (2011) which incorporates the quasi-gold standard for evaluation for improving the performance. Our decision to include the search strings "collaborat*" or "cooperat*" naturally lowered the sensitivity of our search. In our literature review, only those studies were collected and analyzed which refer to pair programming as collaborative or cooperative activity. Because of this specific framework of the search, it is possible, that there exists article that meet our other criteria and would provide additional empirical evidences for our research questions but where not found. Given the overwhelming

amount of research in several research areas, automating the selection of primary studies seems to be a promising approach for further systematic literature reviews (Van Dinter et al., 2021).

Furthermore, for both, RQ1 and RQ2, some inconsistencies of effects across studies have to be stated. Concerning RQ1, direction of effects were mixed for learning outcomes. With respect to RQ2, we found predominantly mixed effects. This may be due to the fact that pair programming in the analyzed studies was implemented in a number of different ways, with differences in, for example, coding tasks, type of course, or length of time spent pair programming. Presumably, this had a significant effect on the results. Furthermore, it is possible that pair programming in the studies often diverged from the ideal collaborative approach, especially if no appropriate instructional guidance for the procedure was provided (Bowman et al., 2020; Coman et al., 2014; Umapathy & Ritzhaupt, 2017). Tasks that are too simple or too brief also might affect pair programming because the available time or the potential value may not be enough for students to engage in collaborative learning (Bowman et al., 2020; Coman et al., 2014). The certainty of evidence thus is limited. This makes the provision of clear recommendations difficult. Moreover, we decided to include in our analysis studies of all types of empirical methods. However, the analyzed studies of descriptive character are not suitable to reveal causal effects. Nevertheless, we consider the inclusion as valuable to – with the necessary caution in interpretation of results – broaden the knowledge on pair programming in higher education.

Conclusion and outlook

This paper has presented the results of a systematic literature review on empirical studies on pair programming. With respect to previous systematic reviews and meta-analyses, our study has three contributions: (1) covering recent articles from the last 10 years, (2) broaden the scope of analysis, in particular to research on instructional design of pair programming, (3) providing recommendations for practitioners that implement pair programming at courses in higher education.

For RQ1, our results overall replicate positive findings stated in earlier literature reviews and meta-analyses. In particular, there is empirical evidence that for novices pair programming is an appropriate method for increasing programming performance. For RQ2, our results show that studies investigated a whole range of different aspects of instructional design, such as pairing of students, procedure of pair programming and instructional guidance. Pairing of students was most frequently researched. However, empirical results on effective instructional design are mixed. Regarding RQ3, we identified additional factors that are relevant for consideration for lecturers, as well as for further studies, namely students' selection criteria for grouping, perceived value of pair programming, participation rates, meaningful communication between partners, and appropriate tasks.

On the whole, the current state of knowledge on pair programming in higher education seems to be somewhat mixed, especially regarding effective instructional design but also regarding effects of pair programming in comparison with solo programming. In this respect, there is a need for further research:

(1) Since empirical results of two studies – one of them large-scale (Bowman et al., 2020) – revealed negative effects of pair programming in comparison with solo programming, further research is needed to investigate under which circumstances pair programming is ineffective, for example, considering potential moderator variables, such as type and complexity of task.

(2) Evidence of an effective instructional design for pair programming is still scarce. There is a need for more studies that systematically study didactic variations in pair programming implementation. One emphasis should be on analyzing effective kinds of instructional guidance to support students' collaboration. The potential different effects on different user groups, for example, novices and experts in programming, have to be considered. On the whole, the influence of potential moderating and mediating variables should be given more attention in pair programming research. We refer, for example, to variables relevant for successful collaborative learning like communication skills, problem-solving skills, or team management skills (Xiong et al., 2015). Fine-grained qualitative observation of pair programming could help to identify obstacles to collaboration in the programming process such as the different perception according to gender (Ying et al., 2019). This could be the basis for further development of instructional design of pair programming.

(3) We recommended monitoring students' pair programming in order to intervene and support if necessary. The application of learning analytics for pair programming in higher education as well as the presentation of results for students and lecturers should be examined in further research.

Although we have identified some shortcomings in the research, we conclude from our results that pair programming is an effective method for learning to program in higher education. However, lecturers who would like to apply pair programming in their classroom should carefully plan the implementation and procedure, as well as the monitoring of students' learning process and the provision of appropriate guidance if necessary.

Disclosure statement

No potential conflict of interest was reported by the author(s).

Funding

This work was partially supported by the German Federal Ministry of Education and Research (Grant number: 16DHB3008).

ORCID

Anja Hawlitschek  <http://orcid.org/0000-0001-8727-2364>

Sandra Schulz  <http://orcid.org/0000-0002-2254-6579>

References

- Ahmad, M., Abd Razak, A. H., Omar, M., Yasin, A., Romli, R., Abdul Mutalib, A., & Zahari, A. S. (2015). The impact of knowledge management in pair programming on program quality. In A. Abraham, A. K. Muda, and Y.-H. Choo (Eds.), *Advances in intelligent systems and computing. Pattern analysis, intelligent security and the internet of things* (pp. 159–168). Springer International Publishing. https://doi.org/10.1007/978-3-319-17398-6_15
- Al-Jarrah, A., Pontelli, E., Zaphiris, P., & Ioannou, A. (2016). On the effectiveness of a collaborative virtual pair-programming environment. In P. Zaphiris, and A. Ioannou (Eds.), *Lecture notes in computer science. Learning and collaboration technologies* (pp. 583–595). Springer. <https://doi.org/10.1007/978-3-319-39483-1>
- Alvarado, C., Lee, C. B., & Gillespie, G. (2014). New CS1 pedagogies and curriculum, the same success factors? In *SIGCSE'14, Proceedings of the 45th ACM technical symposium on computer sciences education* (pp. 379–384). Association for Computing Machinery. <https://doi.org/10.1145/2538862.2538897>
- Anderson, N., & Gegg-Harrison, T. (2013). Learning computer science in the 'comfort zone of proximal development'. In *SIGCSE '13, Proceedings of the 44th ACM technical symposium on computer science education* (pp. 495–500). Association for Computing Machinery. <https://doi.org/10.1145/2445196.2445344>
- Benadé, T., & Liebenberg, J. (2017). Pair programming as a learning method beyond the context of programming. In *CSERC '17, Proceedings of the 6th computer science education research conference* (pp. 48–55). Association for Computing Machinery. <https://doi.org/10.1145/3162087.3162098>
- Boon, M. H., & Thomson, H. (2021). The effect direction plot revisited: Application of the 2019 Cochrane handbook guidance on alternative synthesis methods. *Research Synthesis Methods*, 12(1), 29–33. <https://doi.org/10.1002/jrsm.1458>
- Bowers, C. A., Pharmer, J. A., & Salas, E. (2000). When member homogeneity is needed in work teams. *Small Group Research*, 31(3), 305–327. <https://doi.org/10.1177/104649640003100303>
- Bowman, N. A., Jarratt, L., Culver, K. C., & Segre, A. M. (2019). How prior programming experience affects students' pair programming experiences and outcomes. In *ITiCSE '19, Proceedings of the ACM conference on innovation and technology in computer science education 2019* (pp. 170–175). Association for Computing Machinery. <https://doi.org/10.1145/3304221.3319781>
- Bowman, N. A., Jarratt, L., Culver, K. C., & Segre, A. M. (2020). Pair programming in perspective: Effects on persistence, achievement, and equity in computer science. *Journal of Research on Educational Effectiveness*, 13(4), 1–28. <https://doi.org/10.1080/19345747.2020.1799464>
- Brought, G., McCormick, J., & Wahls, T. (2010). The benefits of pairing by ability. In *SIGCSE '10, Proceedings of the 41st ACM technical symposium on computer science education* (pp. 249–253). Association for Computing Machinery. <https://doi.org/10.1145/1734263.1734348>
- Brought, G., Wahls, T., & Eby, L. M. (2011). The case for pair programming in the computer science classroom. *ACM Transactions on Computing Education*, 11(1), 1–21. <https://doi.org/10.1145/1921607.1921609>
- Bryant, S., Romero, P., & Du Boulay, B. (2006). The collaborative nature of pair programming. In P. Abrahamsson, M. Marchesi, & G. Succi (Eds.), *Extreme programming and agile processes in software engineering* (pp. 53–64). Springer. https://doi.org/10.1007/11774129_6
- Campbell, M., McKenzie, J. E., Sowden, A., Katikireddi, S. V., Brennan, S. E., Ellis, S., Hartmann-Boyce, J., Ryan, R., Shepperd, S., Thomas, J., Welch, V., & Thomson, H. (2020). Synthesis without meta-analysis (SWiM) in systematic reviews: Reporting guideline. *BMJ*, 368, 1–6. <http://dx.doi.org/10.1136/bmj.l6890>
- Celepku, M., & Boyer, K. E. (2018a). The importance of producing shared code through pair programming. In *SIGCSE '18, Proceedings of the 49th ACM technical symposium on computer science education* (pp. 765–770). Association for Computing Machinery. <https://doi.org/10.1145/3159450.3159506>

- Celepku, M., & Boyer, K. E. (2018b). Thematic analysis of students' reflections on pair programming in CS1. In *SIGCSE '18, Proceedings of the 49th ACM technical symposium on computer science education* (pp. 771–776). Association for Computing Machinery. <https://doi.org/10.1145/3159450.3159516>
- Chen, K., & Rea, A. (2018). Do pair programming approaches transcend coding? Measuring agile attitudes in diverse information systems courses. *Journal of Information Systems Education*, 29(2), 53–64. <http://jise.org/Volume29/n2/JISEv29n2p53.html>
- Choi, K. S. (2013). Evaluating gender significance within a pair programming context. In *HICSS '13, Proceedings of the 46th Hawaii international conference on system sciences 2013* (pp. 4817–4825). IEEE Computer Society. <https://doi.org/10.1109/HICSS.2013.209>
- Coman, I. D., Robillard, P. N., Sillitti, A., & Succi, G. (2014). Cooperation, collaboration and pair-programming: Field studies on backup behavior. *Journal of Systems and Software*, 91, 124–134. <https://doi.org/10.1016/j.jss.2013.12.037>
- Cukurova, M., Zhou, Q., Spikol, D., & Landolfi, L. (2020). Modelling collaborative problem-solving competence with transparent learning analytics: Is video data enough? In *LAK '20, Proceedings of the 10th international conference on learning analytics & knowledge (LAK '20)* (pp. 270–275). Association for Computing Machinery. <https://doi.org/10.1145/3375462.3375484>
- de Oliveira, C. M. C., Canedo, E. D., Faria, H., Amaral, L. H. V., & Bonifácio, R. (2018). Improving student's learning and cooperation skills using coding dojos (in the wild!). In *FIE '18, Proceedings of the IEEE frontiers in education conference 2018* (pp. 1–8). IEEE. <https://doi.org/10.1109/FIE.2018.8659056>
- Demir, Ö., & Seferoglu, S. S. (2020a). A comparison of solo and pair programming in terms of flow experience, coding quality, and coding achievement. *Journal of Educational Computing Research*, 58(8), 1–19. <https://doi.org/10.1177/0735633120949788>
- Demir, Ö., & Seferoglu, S. S. (2020b). The effect of determining pair programming groups according to various individual difference variables on group compatibility, flow, and coding performance. *Journal of Educational Computing Research*, 59(1), 1–30. <https://doi.org/10.1177/0735633120949787>
- Dongo, T., Reed, A. H., & O'Hara, M. (2016). Exploring pair programming benefits for MIS majors. *Journal of Information Technology Education-Innovations in Practice*, 15, 223–239. <https://doi.org/10.28945/3625>
- Döring, N., & Bortz, J. (2016). *Forschungsmethoden und Evaluation in den Sozial- und Humanwissenschaften*. Springer. <https://doi.org/10.1007/978-3-642-41089-5>
- Dou, W., & He, W. (2010). Compatibility and requirements analysis of distributed pair programming. In *ECTS '10, Proceedings of the 2nd international workshop on education technology and computer science*, Wuhan, China (pp. 467–470). IEEE. <https://doi.org/10.1109/ETCS.2010.367>
- Dyba, T., Arisholm, E., Sjöberg, D. I. L., Hannay, J. E., & Shull, F. (2007). Are two heads better than one? On the effectiveness of pair programming. *IEEE Software*, 24(5), 12–15. <http://dx.doi.org/10.1109/MS.2007.158>
- Eccles, J. S., Adler, T. F., Futterman, R., Goff, S. B., Kaczala, C. M., Meece, J. L., & Midgley, C. (1983). Expectancies, values, and academic behaviors. In J. T. Spence (Ed.), *Achievement and achievement motivation* (pp. 75–146). W. H. Freeman.
- Edwards, R. L., Stewart, J. K., & Ferati, M. (2010). Assessing the effectiveness of distributed pair programming for an online informatics curriculum. *ACM Inroads*, 1(1), 48–54. <https://doi.org/10.1145/1721933.1721951>
- Estácio, B., Oliveira, R., Marczak, S., Kalinowski, M., Garcia, A., Prikladnicki, R., & Lucena, C. (2015). Evaluating collaborative practices in acquiring programming skills: Findings of a controlled experiment. In *SBES '15, Proceedings of the 29th brazilian symposium on software engineering 2015* (pp. 150–159). IEEE Computer Society. <https://doi.org/10.1109/SBES.2015.24>
- Estácio, B., Valentim, N., Rivero, L., Conte, T., & Prikladnicki, R. (2015). Evaluating the use of pair programming and coding dojo in teaching mockups development: An empirical study. In *HICSS '15, Proceedings of the 48th Hawaii international conference 2015* (pp. 5084–5093). IEEE. <https://doi.org/10.1109/HICSS.2015.602>

- Faja, S. (2014). Evaluating effectiveness of pair programming as a teaching tool in programming courses. *Information Systems Education Journal*, 12(6), 36–45. <https://eric.ed.gov/?id=EJ1140923>
- Garousi, V., Giray, G., Tüzün, E., Catal, C., & Felderer, M. (2019). Aligning software engineering education with industrial needs: A meta-analysis. *Journal of Systems and Software*, 156, 65–83. <https://doi.org/10.1016/j.jss.2019.06.044>
- Gerjets, P., & Scheiter, K. (2003). Goal configurations and processing strategies as moderators between instructional design and cognitive load: Evidence from hypertext-based instruction. *Educational Psychologist*, 38(1), 33–41. https://doi.org/10.1207/S15326985EP3801_5
- Ghorashi, S., & Jensen, C. (2017). Integrating collaborative and live coding for distance education. *Computer*, 50(5), 27–35. <https://doi.org/10.1109/MC.2017.131>
- Gillies, R. M., & Boyle, M. (2010). Teachers' reflections on cooperative learning: Issues of implementation. *Teaching and Teacher Education*, 26(4), 933–940. <https://doi.org/10.1016/j.tate.2009.10.034>
- Gillies, R. M., & Boyle, M. (2011). Teachers' reflections of cooperative learning (CL): A two-year follow-up. *Teaching Education*, 22(1), 63–78. <https://doi.org/10.1080/10476210.2010.538045>
- Goel, S., & Kathuria, V. (2010). A novel approach for collaborative pair programming. *Journal of Information Technology Education*, 9, 183–196. <https://doi.org/10.28945/1290>
- Govender, D. W., & Govender, T. P. (2014). Using a collaborative learning technique as a pedagogic intervention for the effective teaching and learning of a programming course. *Mediterranean Journal of Social Sciences*, 5(20), 1077–1086. <https://doi.org/10.5901/mjss.2014.v5n20p1077>
- Hannay, J. E., Dybå, T., Arisholm, E., & Sjøberg, D. I. K. (2009). The effectiveness of pair programming: A meta-analysis. *Information and Software Technology*, 51(7), 1110–1122. <https://doi.org/10.1016/j.infsof.2009.02.001>
- Harsley, R., Di Eugenio, B., Green, N., & Fossati, D. (2017). Enhancing an intelligent tutoring system to support student collaboration: Effects on learning and behavior. In E. Andre, R. Baker, X. Hu, M. M. T. Rodrigo, & B. DuBoulay (Eds.), *Artificial intelligence in education* (pp. 519–522). Springer. https://doi.org/10.1007/978-3-319-61425-0_54
- Harsley, R., Fossati, D., Di Eugenio, B., & Green, N. (2017). Interactions of individual and pair programmers with an intelligent tutoring system for computer science. In *SIGCSE '17, Proceedings of the ACM SIGCSE technical symposium on computer science education 2017* (pp. 285–290). Association for Computing Machinery. <https://doi.org/10.1145/3017680.3017786>
- Hattie, J., & Timperley, H. (2007). The power of feedback. *Review of Educational Research*, 77(1), 81–112. <https://doi.org/10.3102/003465430298487>
- Jarratt, L., Bowman, N. A., Culver, K. C., & Segre, A. M. (2019). A large-scale experimental study of gender and pair composition in pair programming. In *ITiCSE '19, Proceedings of the ACM conference on innovation and technology in computer science education 2019* (pp. 176–181). Association for Computing Machinery. <https://doi.org/10.1145/3304221.3319782>
- Jermann, P., & Dillenbourg, P. (2008). Group mirrors to support interaction regulation in collaborative problem solving. *Computers & Education*, 51(1), 279–296. <https://doi.org/10.1016/j.compedu.2007.05.012>
- Johnson, D. W., & Johnson, R. T. (2009). An educational psychology success story: Social interdependence theory and cooperative learning. *Educational Researcher*, 38(5), 365–379. <https://doi.org/10.3102/0013189X09339057>
- Jurado, F., Molina, A. I., Redondo, M. A., & Ortega, M. (2013). Cole-programming: Shaping collaborative learning support in eclipse. *IEEE Revista Iberoamericana De Tecnologías Del Aprendizaje*, 8(4), 153–162. <https://doi.org/10.1109/RITA.2013.2284953>
- Kaendler, C., Wiedmann, M., Rummel, N., & Spada, H. (2015). Teacher competencies for the implementation of collaborative learning in the classroom: A framework and research review. *Educational Psychology Review*, 27(3), 505–536. <https://doi.org/10.1007/s10648-014-9288-9>
- Karthikeyan, K., Ahmed, I., & Jayalakshmi, J. (2018). Pair programming for software engineering education: An empirical study. *The International Arab Journal of Information Technology (IAJIT)*, 15(2), 246–255. <https://doi.org/10.1109/TSE.2010.59>

- Kavitha, R. K., & Ahmed, M. S. I. (2015). Knowledge sharing through pair programming in learning environments: An empirical study. *Education and Information Technologies*, 20(2), 319–333. <https://doi.org/10.1007/s10639-013-9285-5>
- Kitchenham, B. A., & Charters, S. (2007). *Guidelines for performing systematic literature reviews in software engineering*, Technical Report EBSE-2007-01, School of Computer Science and Mathematics, Keele University.
- Kongcharoen, C., Hwang, W.-Y., & Ghinea, G. (2017). Synchronized pair configuration in virtualization-based lab for learning computer networks. *Educational Technology & Society*, 20(3), 54–68. <http://www.jstor.org/stable/26196119>
- Kori, K., Pedaste, M., Leijen, Ä., & Tönnisson, E. (2016). The role of programming experience in ICT students' learning motivation and academic achievement. *International Journal of Information and Education Technology*, 6(5), 331–337. <https://doi.org/10.7763/IJiet.2016.V6.709>
- Kori, K., Pedaste, M., Tonisson, E., Palts, T., Altin, H., Rantsus, R., Sell, R., Murtazin, K., & Ruutmann, T. (2015). First-year dropout in ICT studies. In *EDUCON '15, Proceedings of IEEE global engineering education conference 2015* (pp. 437–445). IEEE. <https://doi.org/10.1109/EDUCON.2015.7096008>
- Kreijns, K., Kirschner, P. A., & Jochems, W. (2003). Identifying the pitfalls for social interaction in computer-supported collaborative learning environments: A review of the research. *Computers in Human Behavior*, 19(3), 335–353. [https://doi.org/10.1016/S0747-5632\(02\)00057-2](https://doi.org/10.1016/S0747-5632(02)00057-2)
- Lewis, C. M., Titterton, N., & Clancy, M. (2012). Using collaboration to overcome disparities in Java experience. In *ICER '12, Proceedings of the 9th annual international conference on international computing education research* (pp. 79–86). Association for Computing Machinery. <https://doi.org/10.1145/2361276.2361292>
- Lou, Y., Abrami, P. C., & d'Apollonia, S. (2001). Small group and individual learning with technology: A meta-analysis. *Review of Educational Research*, 71(3), 449–521. <https://doi.org/10.3102/00346543071003449>
- Maguire, P., Maguire, R., Hyland, P., & Marshall, P. (2014). Enhancing collaborative learning using pair programming: Who benefits? *All Ireland Journal of Teaching and Learning in Higher Education*, 6(2), 1411–1419. <https://ojs.aishe.org/index.php/aishe-j/article/view/141>
- Mayer, R. E. (2004). Should there be a three-strikes rule against pure discovery learning? The case for guided methods of instruction. *The American Psychologist*, 59(1), 14–19. <https://doi.org/10.1037/0003-066X.59.1.14>
- Mayer, R. E. (2011). Multimedia learning and games. In S. Tobias & J. D. Fletcher (Eds.), *Computer games and instruction* (pp. 281–305). Information Age Publishing.
- Mayer, R. E., & Johnson, C. I. (2010). Adding instructional features that promote learning in a game-like environment. *Journal of Educational Computing Research*, 42(3), 241–265. <https://doi.org/10.2190/EC.42.3.a>
- McChesney, I. (2016). Three years of student pair programming: Action research insights and outcomes. In *SIGCSE '16, Proceedings of the 47th ACM technical symposium on computing science education* (pp. 84–89). Association for Computing Machinery. <https://doi.org/10.1145/2839509.2844565>
- Nawahdah, M., & Taji, D. (2016). Investigating students' behavior and code quality when applying pair-programming as a teaching technique in a Middle Eastern society. In *EDUCON '16, Proceedings of IEEE global engineering education conference 2016* (pp. 32–39). IEEE. <https://doi.org/10.1109/EDUCON.2016.7474527>
- Nokes-Malach, T. J., Richey, J. E., & Gadgil, S. (2015). When is it better to learn together? Insights from research on collaborative learning. *Educational Psychology Review*, 27(4), 645–656. <https://doi.org/10.1007/s10648-015-9312-8>
- O'Donnell, C., Buckley, J., Mahdi, A., Nelson, J., & English, M. (2015). Evaluating pair-programming for non-computer science major students. In *SIGCSE '15, Proceedings of the 46th ACM technical symposium on computer science education* (pp. 569–574). Association for Computing Machinery. <https://doi.org/10.1145/2676723.2677289>

- Olivares, O. J. (2008). Collaborative vs. cooperative learning: The instructor's role in computer supported collaborative learning. In K. L. Orvis & A. L. R. Lassiter (Eds.), *Computer-supported collaborative learning: Best practices and principles for instructors* (pp. 20–39). IGI global.
- Page, M. J., McKenzie, J. E., Bossuyt, P. M., Boutron, I., Hoffmann, T. C., Mulrow, C. D., Shamseer, L., Tetzlaff, J. M., Akl, E. A., Brennan, S. E., Chou, R., Glanville, J., Grimshaw, J. M., Hróbjartsson, A., Lalu, M. M., Li, T., Loder, E. W., Mayo-Wilson, E., McDonald, S., , and Moher, D. (2021). The PRISMA 2020 statement: An updated guideline for reporting systematic reviews. *BMJ*, 372(8284), n71. <https://doi.org/10.1136/bmj.n71>
- Radermacher, A., Walia, G., & Rummelt, R. (2012). Improving student learning outcomes with pair programming. In *ICER '12, Proceedings of the ninth annual international conference on international computing education research* (pp. 87–92). Association for Computing Machinery. <https://doi.org/10.1145/2361276.2361294>
- Reckinger, S., & Hughes, B. (2020). Strategies for implementing in-class, active, programming assessments: A multi-level model. In *SIGCSE '20, Proceedings of the 51st ACM technical symposium on computer science education* (pp. 454–460). Association for Computing Machinery. <https://doi.org/10.1145/3328778.3366850>
- Reigeluth, C. M. (1983). Instructional design: What is it and why is it. In C. M. Reigeluth (Ed.), *Instructional-design theories and models: An overview of their current status* (pp. 3–36). Erlbaum.
- Reigeluth, C. M., & Carr-Chellman, A. A. (2009). Understanding instructional theory. In C. M. Reigeluth & A. A. Carr-Chellman (Eds.), *Instructional-design theories and models. Building a common knowledge base* (pp. 3–26). Taylor and Francis.
- Rodrigo, M. M. T. (2017). Exploratory analysis of discourses between students engaged in a debugging task. In *ICCE '17, Proceedings of the 25th international conference on computers in education* (pp. 198–203). Asia-Pacific Society for Computers in Education.
- Rodriguez, F. J., & Boyer, K. E. (2015). Discovering individual and collaborative problem-solving modes with hidden Markov models. In C. Conati, N. Heffernan, A. Mitrovic, & M. F. Verdejo (Eds.), *Artificial intelligence in education* (pp. 408–418). Springer. https://doi.org/10.1007/978-3-319-19773-9_41
- Rodríguez, F. J., Price, K. M., & Boyer, K. E. (2017). Exploring the pair programming process: Characteristics of effective collaboration. In *SIGCSE '17, Proceedings of the ACM SIGCSE technical symposium on computer science education 2017* (pp. 507–512). Association for Computing Machinery. <https://doi.org/10.1145/3017680.3017748>
- Rong, G., Zhang, H., Xie, M., & Shao, D. (2012). Improving PSP education by pairing: An empirical study. In *ICSE '12, Proceedings of the 34th international conference on software engineering 2012* (pp. 1245–1254). IEEE. <https://doi.org/10.1109/ICSE.2012.6227018>
- Rummel, N., & Spada, H. (2005). Learning to collaborate: An instructional approach to promoting collaborative problem solving in computer-mediated settings. *Journal of the Learning Sciences*, 14 (2), 201–241. https://doi.org/10.1207/s15327809jls1402_2
- Ruys, I., Keer, H., & Aelterman, A. (2012). Examining pre-service teacher competence in lesson planning pertaining to collaborative learning. *Journal of Curriculum Studies*, 44(3), 349–379. <https://doi.org/10.1080/00220272.2012.675355>
- Saab, N., van Joolingen, W. R., & van Hout-Wolters, B. H. A. (2007). Supporting communication in a collaborative discovery learning environment: The effect of instruction. *Instructional Science*, 35 (1), 73–98. <https://doi.org/10.1007/s11251-006-9003-4>
- Salleh, N., Mendes, E., & Grundy, J. (2011). Empirical studies of pair programming for CS/SE teaching in higher education: A systematic literature review. *IEEE Transactions on Software Engineering*, 37 (4), 509–525. <https://doi.org/10.1109/TSE.2010.59>
- Salleh, N., Mendes, E., & Grundy, J. (2014). Investigating the effects of personality traits on pair programming in a higher education setting through a family of experiments. *Empirical Software Engineering*, 19(3), 714–752. <https://doi.org/10.1007/s10664-012-9238-4>
- Satratzemi, M., Xinogalos, S., Tsompanoudi, D., & Karamitopoulos, L. (2018). Examining student performance and attitudes on distributed pair programming. *Scientific Programming*, 2018, 1–8. <https://doi.org/10.1155/2018/6523538>

- Schneider, M., & Preckel, F. (2017). Variables associated with achievement in higher education: A systematic review of meta-analyses. *Psychological Bulletin*, 143(6), 565–600. <https://doi.org/10.1037/bul0000098>
- Sennett, J., & Sherriff, M. (2010). Compatibility of partnered students in computer science education. In *SIGCSE '10, Proceedings of the 41st ACM technical symposium on computer science education* (pp. 244–248). Association for Computing Machinery. <https://doi.org/10.1145/1734263.1734347>
- Sweller, J., van Merriënboer, J. J. G., & Paas, F. G. W. C. (1998). Cognitive architecture and instructional design. *Educational Psychology Review*, 10(3), 251–296. <https://doi.org/10.1023/A:1022193728205>
- Tsompanoudi, D., Satratzemi, M., & Xinogalos, S. (2013). Exploring the effects of collaboration scripts embedded in a distributed pair programming system. In *Proceedings of the 13th ACM/IEEE-CS joint conference on digital libraries* (pp. 225–230). Association for Computing Machinery. <https://doi.org/10.1145/2462476.2462500>
- Tsompanoudi, D., Satratzemi, M., & Xinogalos, S. (2015). Distributed pair programming using collaboration scripts: An educational system and initial results. *Informatics in Education*, 14(2), 291–314. <https://doi.org/10.15388/infedu.2015.17>
- Tsompanoudi, D., Satratzemi, M., & Xinogalos, S. (2016). Evaluating the effects of scripted distributed pair programming on student performance and participation. *IEEE Transactions on Education*, 59(1), 24–31. <https://doi.org/10.1109/TE.2015.2419192>
- Tsompanoudi, D., Satratzemi, M., Xinogalos, S., & Karamitopoulos, L. (2019). An empirical study on factors related to distributed pair programming. *International Journal of Engineering Pedagogy*, 9(2), 61–81. <https://doi.org/10.3991/ijep.v9i2.9947>
- Umapathy, K., & Ritzhaupt, A. D. (2017). A meta-analysis of pair-programming in computer programming courses. *ACM Transactions on Computing Education*, 17(4), 1–13. <https://doi.org/10.1145/2996201>
- Urai, T., Umezawa, T., & Osawa, N. (2015). Enhancements to support functions of distributed pair programming based on action analysis. In *ITICSE '15, Proceedings of the ACM conference on innovation and technology in computer science education 2015* (pp. 177–182). Association for Computing Machinery. <https://doi.org/10.1145/2729094.2742616>
- van Boxtel, C., van der Linden, J., & Kanselaar, G. (2000). Collaborative learning tasks and the elaboration of conceptual knowledge. *Learning and Instruction*, 10(4), 311–330. [https://doi.org/10.1016/S0959-4752\(00\)00002-5](https://doi.org/10.1016/S0959-4752(00)00002-5)
- van Dinter, R., Tekinerdogan, B., & Catal, C. (2021). Automation of systematic literature reviews: A systematic literature review. *Information and Software Technology*, 136, 106589. <https://doi.org/10.1016/j.infsof.2021.106589>
- Viberg, O., Hatakka, M., Bälter, O., & Mavroudi, A. (2018). The current landscape of learning analytics in higher education. *Computers in Human Behavior*, 89, 98–110. <https://doi.org/10.1016/j.chb.2018.07.027>
- Webster, J., & Watson, R. T. (2002). Analyzing the past to prepare for the future: Writing a literature review. *MIS Quarterly*, 26(2), iii–xiii. <http://www.jstor.org/stable/4132319>
- Xinogalos, S., Malliarakis, C., Tsompanoudi, D., & Satratzemi, M. (2015). Microworlds, games and collaboration: Three effective approaches to support novices in learning programming. In *BCI '15, Proceedings of the 7th Balkan conference on informatics conference* (pp. 1–8). Association for Computing Machinery. <https://doi.org/10.1145/2801081.2801094>
- Xinogalos, S., Satratzemi, M., Chatzigeorgiou, A., & Tsompanoudi, D. (2017). Student perceptions on the benefits and shortcomings of distributed pair programming assignments. In *EDUCON '17, Proceedings of the IEEE global engineering education conference 2017* (pp. 1512–1520). IEEE. <https://doi.org/10.1109/EDUCON.2017.7943050>
- Xinogalos, S., Satratzemi, M., Chatzigeorgiou, A., & Tsompanoudi, D. (2019). Factors affecting students' performance in distributed pair programming. *Journal of Educational Computing Research*, 57(2), 513–544. <https://doi.org/10.1177/0735633117749432>

- Xiong, Y., So, H. J., & Toh, Y. (2015). Assessing learners' perceived readiness for computer-supported collaborative learning (CSCCL): A study on initial development and validation. *Journal of Computing in Higher Education*, 27(3), 215–239. <https://doi.org/10.1007/s12528-015-9102-9>
- Yang, Y.-F., Lee, C.-I., & Chang, C.-K. (2016). Learning motivation and retention effects of pair programming in data structures courses. *Education for Information*, 32(3), 249–267. <https://doi.org/10.3233/EFI-160976>
- Ying, K. M., Pezzullo, L. G., Ahmed, M., Crompton, K., Blanchard, J., & Boyer, K. E. (2019). In their own words: Gender differences in student perceptions of pair programming. In *SIGCSE '19, Proceedings of the 50th ACM technical symposium on computer science education* (pp. 1053–1059). Association for Computing Machinery. <https://doi.org/10.1145/3287324.3287380>
- Zacharis, N. Z. (2011). Measuring the effects of virtual pair programming in an introductory programming Java course. *IEEE Transactions on Education*, 54(1), 168–170. <https://doi.org/10.1109/TE.2010.2048328>
- Zarb, M., Hughes, J., & Richards, J. (2013). Industry-inspired guidelines improve students' pair programming communication. In *ITiCSE '13, Proceedings of the 18th ACM conference on innovation and technology in computer science education* (pp. 135–140). Association for Computing Machinery. <https://doi.org/10.1145/2462476.2462504>
- Zarb, M., Hughes, J., & Richards, J. (2014). Evaluating industry-inspired pair programming communication guidelines with undergraduate students. In *SIGCSE'14, Proceedings of the 45th ACM technical symposium on computer sciences education* (pp. 361–366). Association for Computing Machinery. <https://doi.org/10.1145/2538862.2538980>
- Zarb, M., Hughes, J., & Richards, J. (2015). Further evaluations of industry-inspired pair programming communication guidelines with undergraduate students. In *SIGCSE '15, Proceedings of the 46th ACM technical symposium on computer science education* (pp. 314–319). Association for Computing Machinery. <https://doi.org/10.1145/2676723.2677241>
- Zhang, H., Babar, M. A., & Tell, P. (2011). Identifying relevant studies in software engineering. *Information and Software Technology*, 53(6), 625–637. <https://doi.org/10.1016/j.infsof.2010.12.010>
- Zhang, Y., Yang, Y., Yang, Y., & Dou, W. (2017). Research on pair learning method and pattern based on pair programming. In *CETCU 2017, Proceedings of the 3rd conference on education and teaching in colleges and universities* (pp. 58–61), Atlantis Press. <https://doi.org/10.2991/cetcu-17.2017.14>